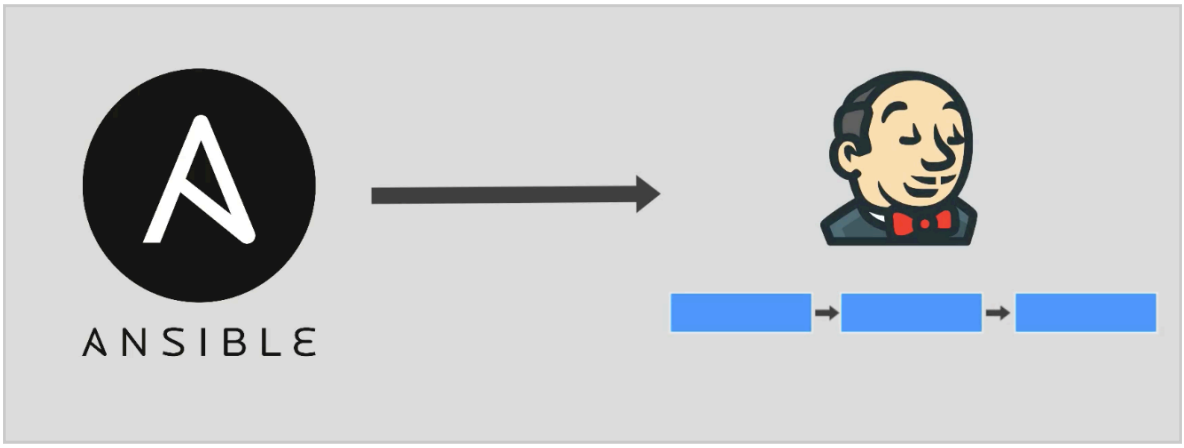
Module 15 - chapter 21
Project: Run Ansible from Jenkins Pipeline - Part 1

Ansible Integration in Jenkins

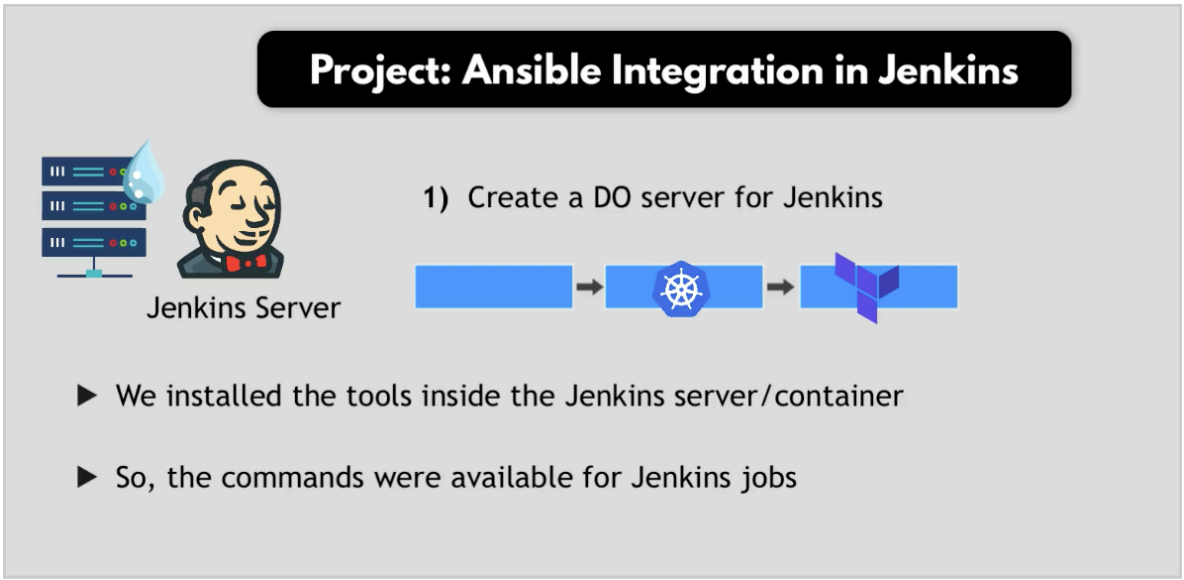
Overview

How to execute ansible playbooks from Jenkins Pipeline:



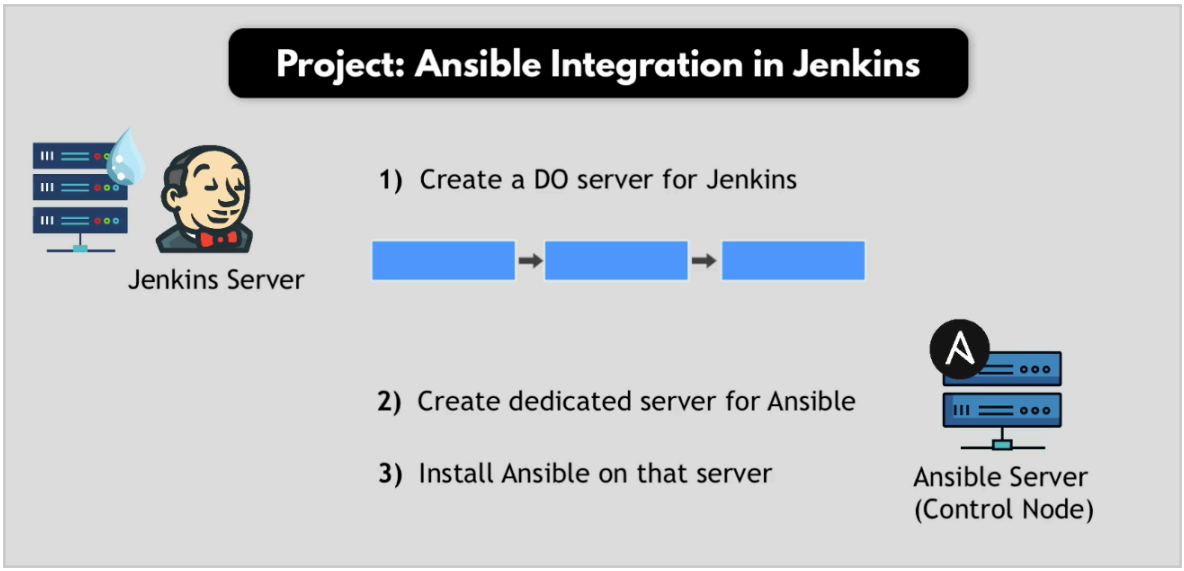
Implementation steps:
1. Create a DO (DigitalOcean) droplet

Previously we learn that if we wanted to use a tool in jenkins we need to install it there first for example kubectl.



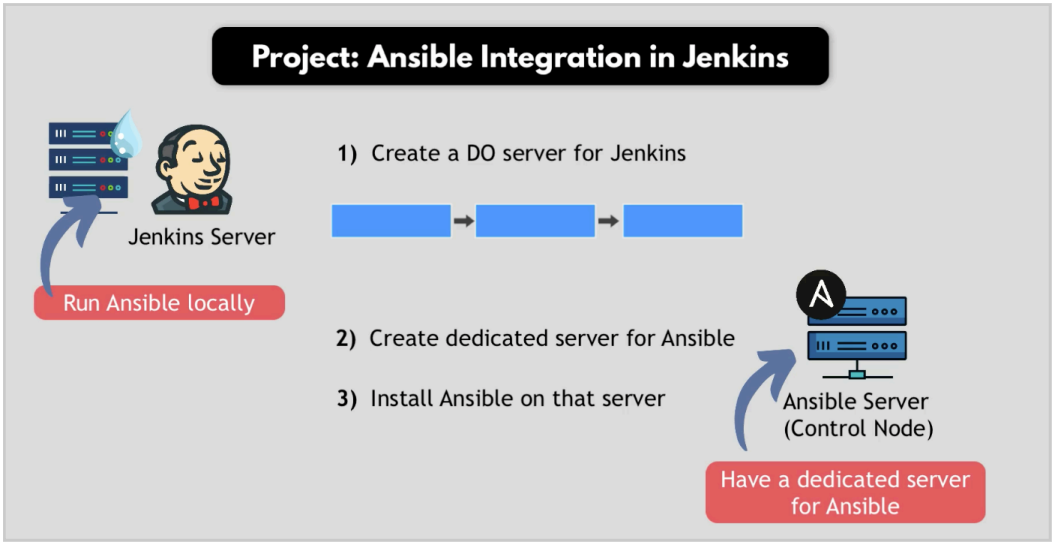
But for ansible it will be different, instead of installing ansible in the jenkins container and making it available in the Jenkins Server we will create a dedicated server for ansible and install ansible there.

2. Create dedicated server for ansible
3. Install ansible on that server

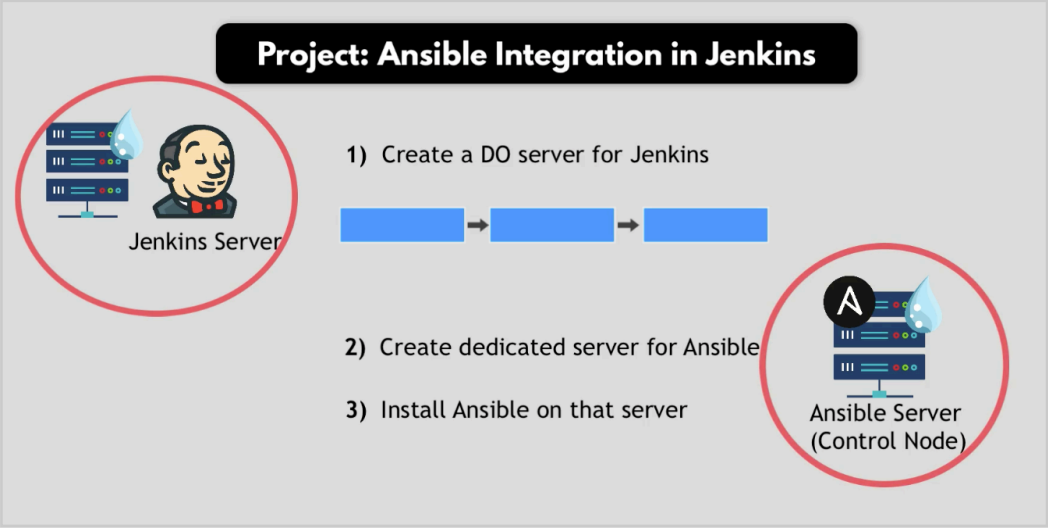


We can run ansible locally for example in our laptop or in Jenkis directly.

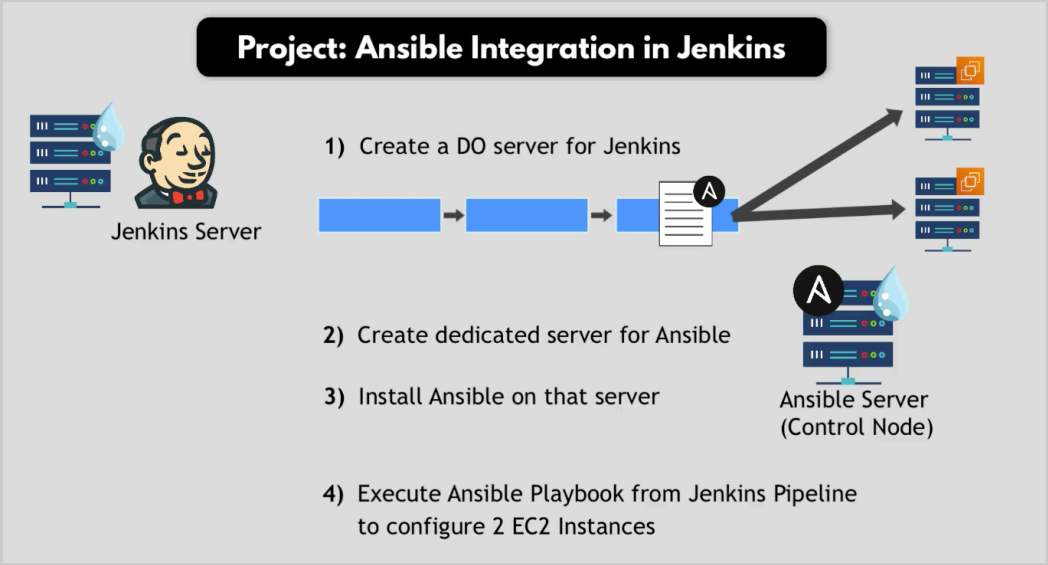
Or as a common practice we will have a dedicated server for ansible and install ansible there and execute ansible commands from that server.



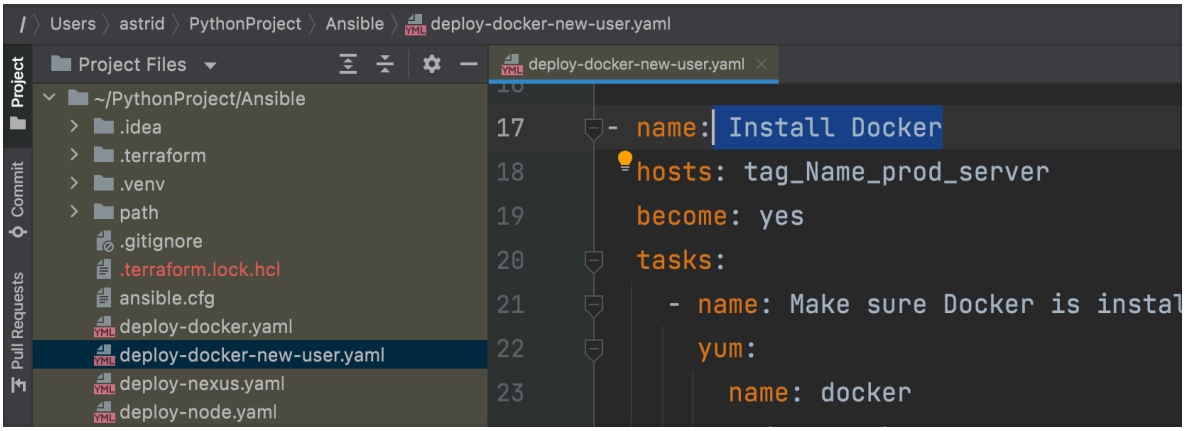
So we will have separate droplets



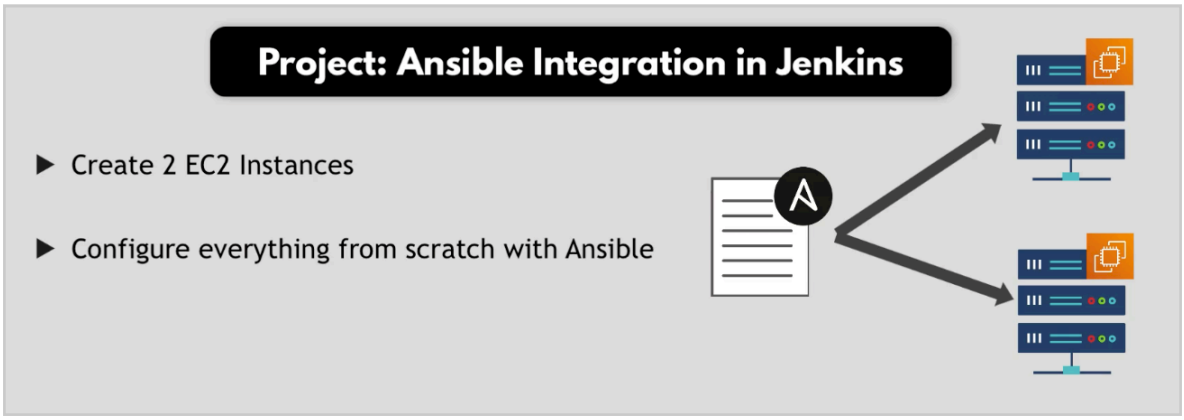
4. Then we want to execute an Ansible playbook from Jenkins Pipeline to configure two EC2 instances, it will install docker and docker-compose on the EC2 instances.



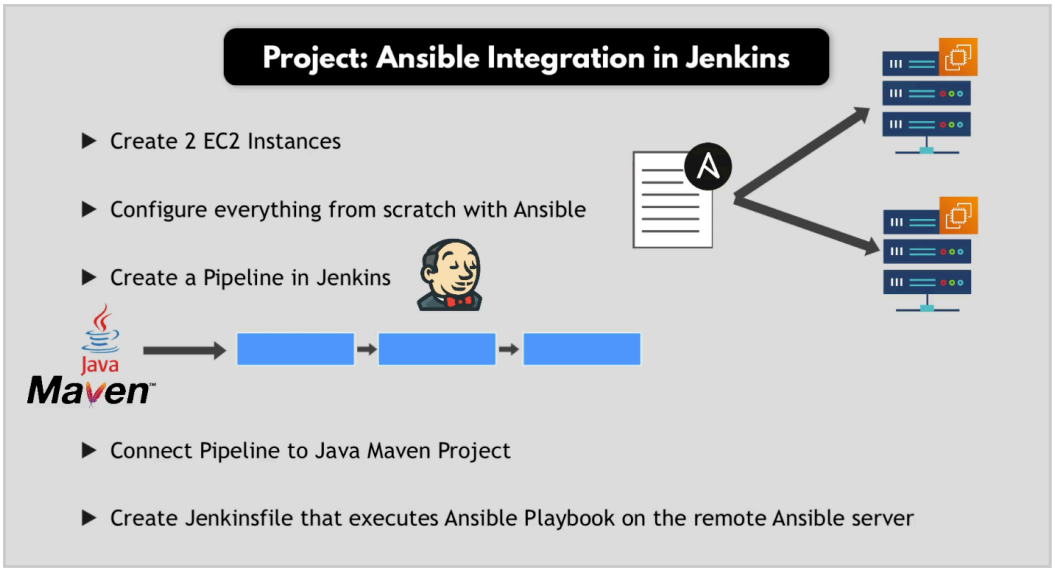
We already have playbook to configure docker and docker-compose called ‘deploy-docker-new-user.yaml’



So we will create two EC2 instances and Ansible will configure them

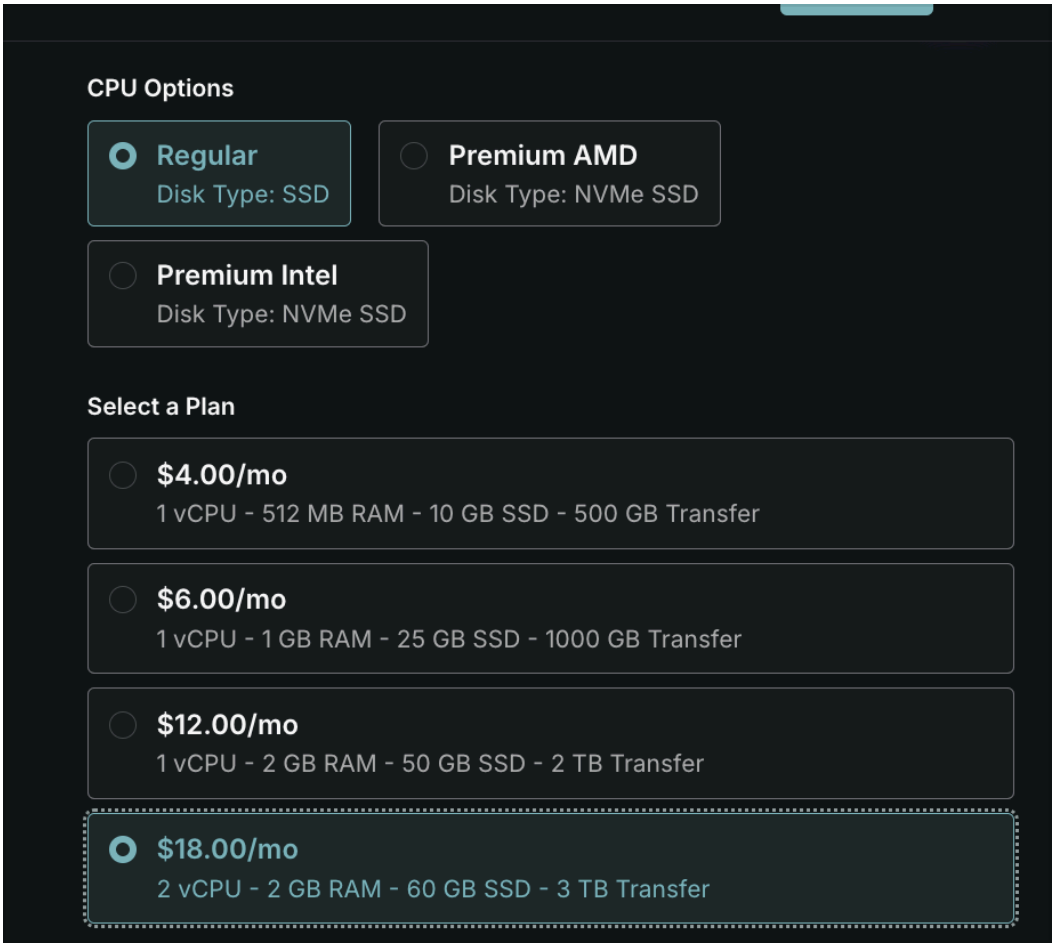


Then we will create a pipeline in Jenkins and connect the pipeline to a Java Maven project and finally create a Jenkinsfile that executes Ansible Playbook on the remote Ansible server

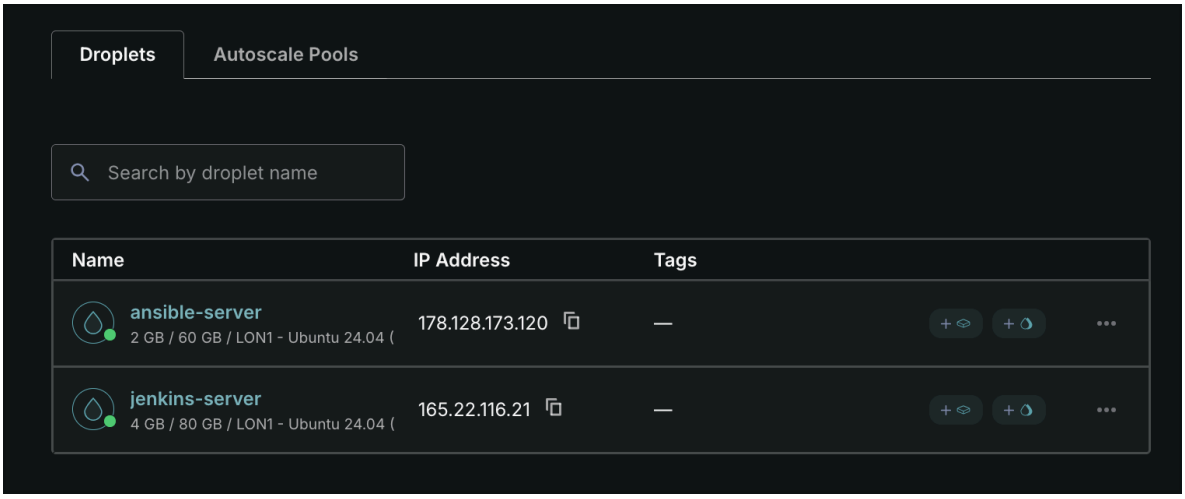


Prepare Ansible Control Node

Create droplet for Ansible -> the ansible control node



Server is ready



Now ssh into the server and install ansible on it

-> ssh root@178.128.173.120
-> apt update

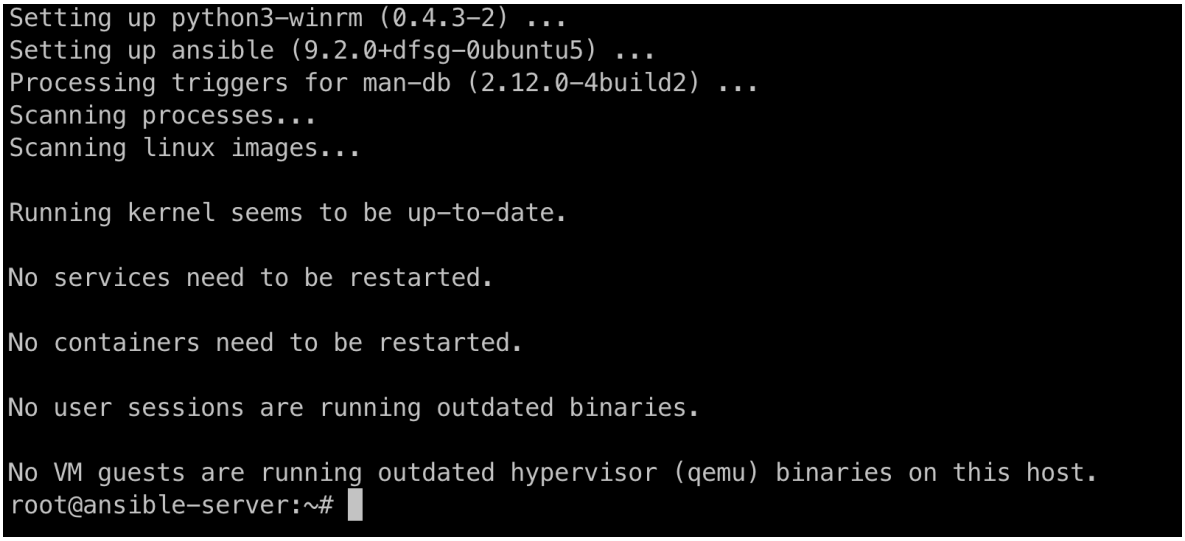
```
root@ansible-server:~# apt update
Hit:1 http://mirrors.digitalocean.com/ubun
Get:2 http://mirrors.digitalocean.com/ubun
```

-> ansible

```
root@ansible-server:~# ansible
Command 'ansible' not found, but can be installed with:
apt install ansible-core
root@ansible-server:~#
```

-> apt install ansible-core

```
root@ansible-server:~# apt install ansible-core
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ansible python3-argcomplete python3-dnspython python3-kerbero
  python3-requests-ntlm python3-resolvelib python3-selinux pyth
Suggested packages:
  cowsay sshpass python3-trio python3-aioquic python3-h2 python
The following NEW packages will be installed:
  ansible ansible-core python3-argcomplete python3-dnspython py
  python3-passlib python3-requests-ntlm python3-resolvelib pyth
0 upgraded, 15 newly installed, 0 to remove and 95 not upgraded
Need to get 19.5 MB of archives.
After this operation, 315 MB of additional disk space will be u
Do you want to continue? [Y/n] y
Get:1 http://mirrors.digitalocean.com/ubuntu noble/universe amd
Get:2 http://mirrors.digitalocean.com/ubuntu noble/main amd64 p
```



Also, we need to install a python module for AWS -> boto3, ansible is based on python. Some of the tasks we run in ansible need the python module for AWS which ansible will use in the background

Why Ansible needs it

Ansible itself is a Python program, but it doesn't know how to talk to any specific API (AWS, Kubernetes, etc.) natively. Each provider's modules (e.g. the amazon.aws / community.aws collections) are thin Python wrappers that, under the hood, call boto3 — AWS's official Python SDK — to actually make the HTTP requests to the AWS API (create an EC2 instance, read a VPC, etc.).

So when a task like amazon.aws.ec2_instance runs, the sequence is:

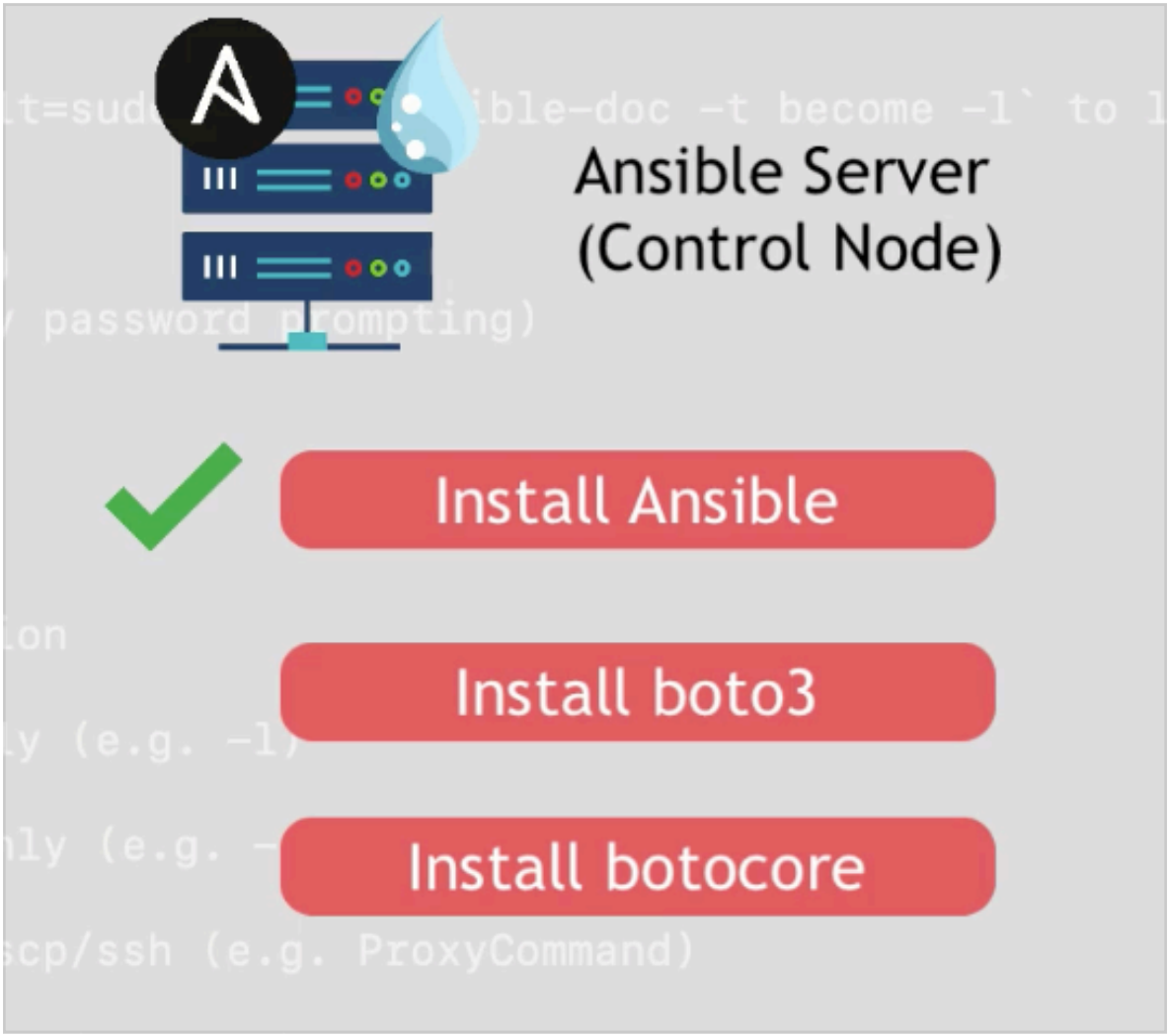
Ansible task → AWS collection module (Python) → boto3 → AWS API

If boto3 isn't installed in the Python environment Ansible is using, the module fails immediately with an error like "msg": "boto3 required for this module" — it's not an Ansible bug, it's a missing dependency for the underlying library the

module calls into.

(You may also see botocore listed alongside it — that's the lower-level library boto3 itself is built on, handling auth, request signing, retries. Installing boto3 usually pulls botocore in as a dependency automatically.)

As we will configure EC2 instances with Ansible we will use boto3 and botocore, so we need to install them



-> apt install python3-boto3

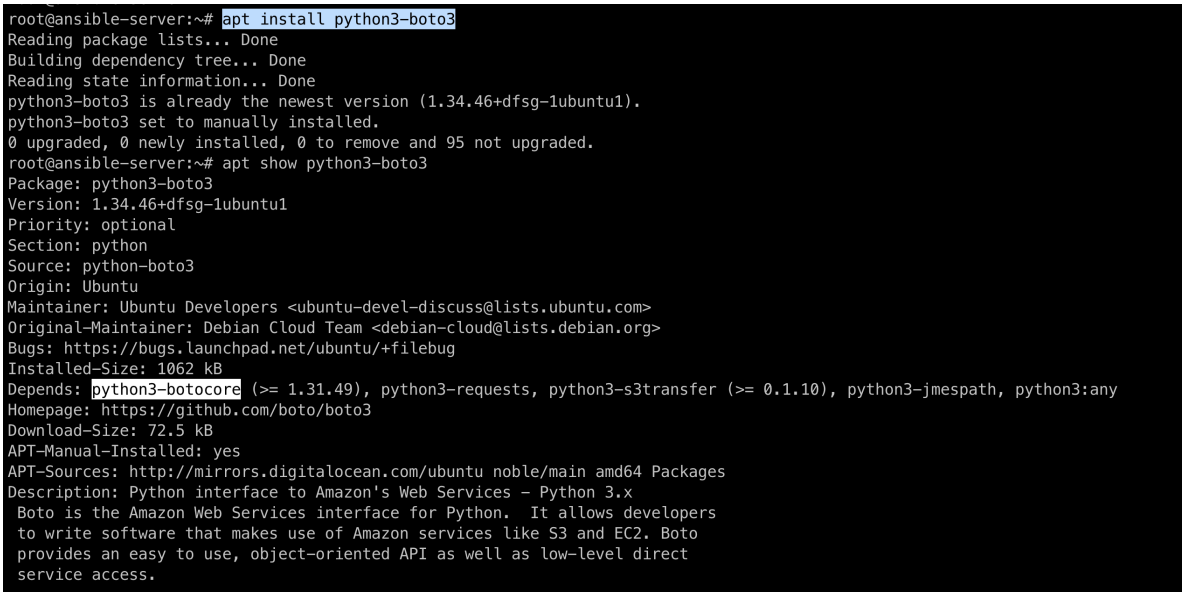
```
root@ansible-server:~# apt install python3-boto3
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
python3-boto3 is already the newest version (1.34.46+dfsg-1ubuntu1).
python3-boto3 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 95 not upgraded.
root@ansible-server:~#
```

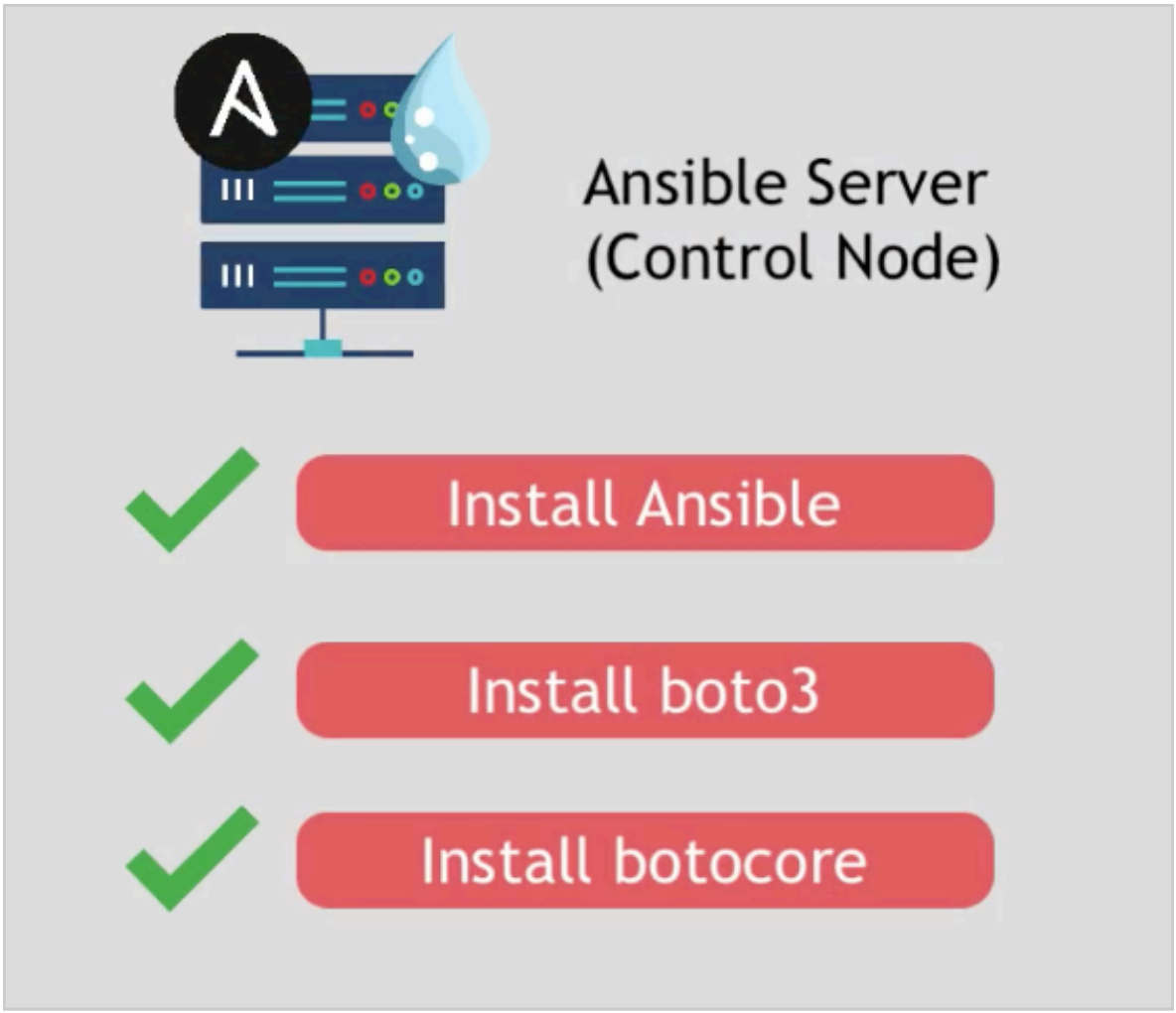
apt didn't need to do anything about it this run.

Why

python3-boto3 is a Debian/Ubuntu package that depends on python3-botocore (declared in its package metadata). apt resolves and installs dependencies automatically the first time you install a package — but in your case, boto3 was "already the newest version," meaning it (and therefore botocore, pulled in as its dependency) was already installed before you ran this command, likely baked into the droplet image or installed earlier. Since nothing needed to change, apt's output only reports "0 upgraded, 0 newly installed, 0 to remove" — it doesn't re-announce dependencies that are already satisfied.

-> apt show python3-boto3

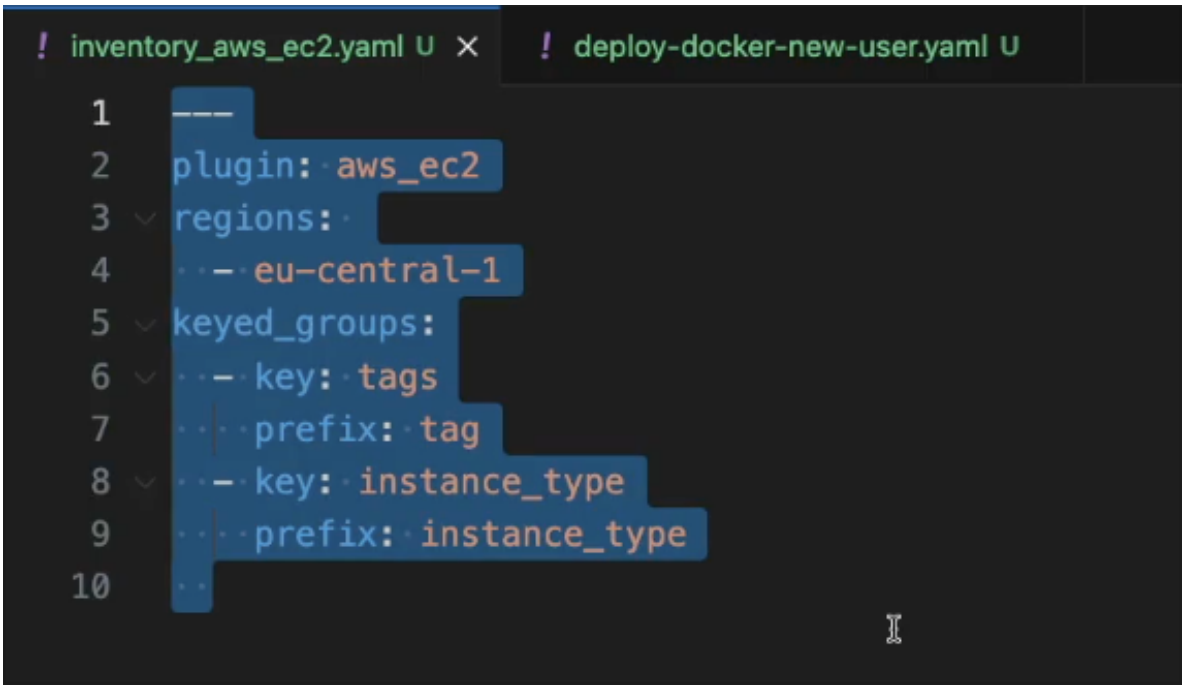




Now we need AWS credentials:

boto3 also needs AWS credentials to actually authenticate — same as the AWS CLI. Either:
- ~/.aws/credentials on the droplet, or
- environment variables (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY), or
- an IAM role if the droplet were an EC2 instance (not applicable here since it's a Droplet, i.e. DigitalOcean, not AWS).

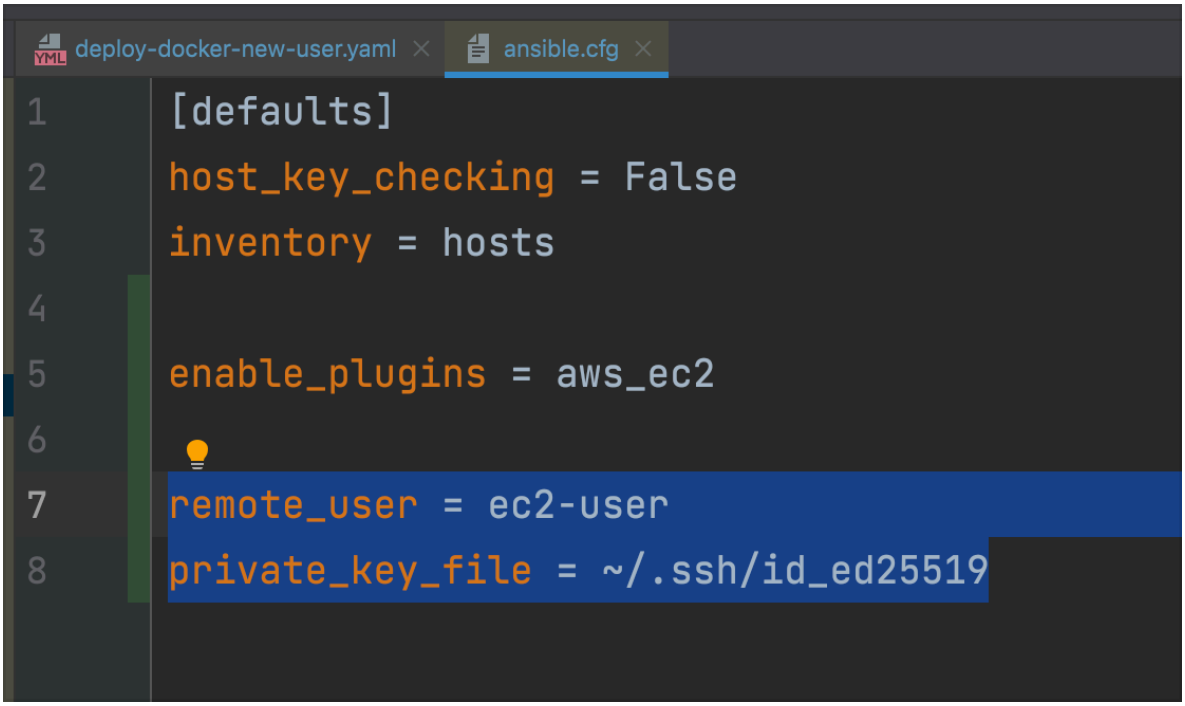
We will be using a dynamic inventory for the docker playbook



So instead of hardcoding the EC2 instances IP addresses we get them dynamically from AWS and we need AWs credentials for that dynamic inventory to work.

The inventory will try to fetch the EC2 instances and for that it needs to authenticate (when we run this in my laptop in Module 15 - chapter 19 the auth info was already there due to -> aws configure - which wrote my access key/ secret into **~/.aws/credentials**) but in this case the droplet doesn't have the credentials stored anywhere.

Don't confuse the AWS credentials with remote_user and private_key_file from Module 15 - chapter 19 notes , that configuration is to ssh into each host not to authenticate to AWS.

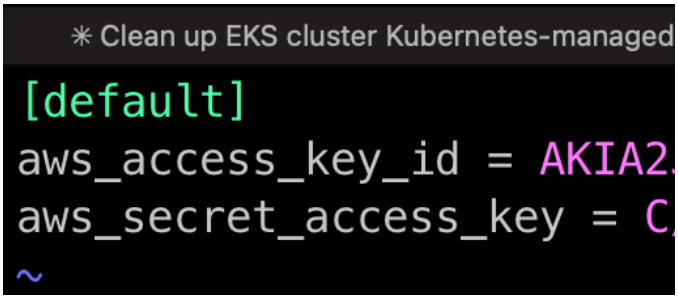


So we need to create the .aws folder inside the ansible server
-> mkdir .aws

```
root@ansible-server:~# pwd
/root
root@ansible-server:~# ls
root@ansible-server:~# mkdir .aws
root@ansible-server:~# ls
root@ansible-server:~# ls -a
.  ..  .aws  .bashrc  .cache  .cloud-locale-test.skip  .profile  .ssh  .wget-hsts
root@ansible-server:~#
```

Now let's create the file credentials inside .aws directory
-> cd .aws
-> vim credentials

And I can copy the contents of my laptop's credentials file into this file:



```
root@ansible-server:~# cd .aws/
root@ansible-server:~/.aws# vim credentials
root@ansible-server:~/.aws# cat credentials
[default]
aws_access_key_id = [redacted]
aws_secret_access_key = [redacted]
root@ansible-server:~/.aws#
```

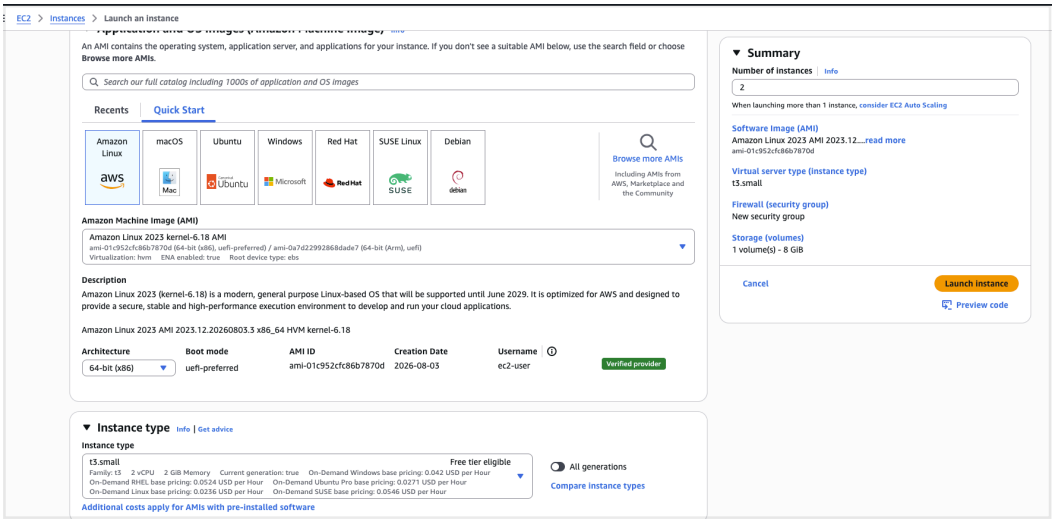
So we have boto library ready and the credentials ready for when we need it to use dynamic inventory because the plugin will need to authenticate with AWS

```
root@ansible-server:~# ls .aws/credentials
.aws/credentials
root@ansible-server:~#
```

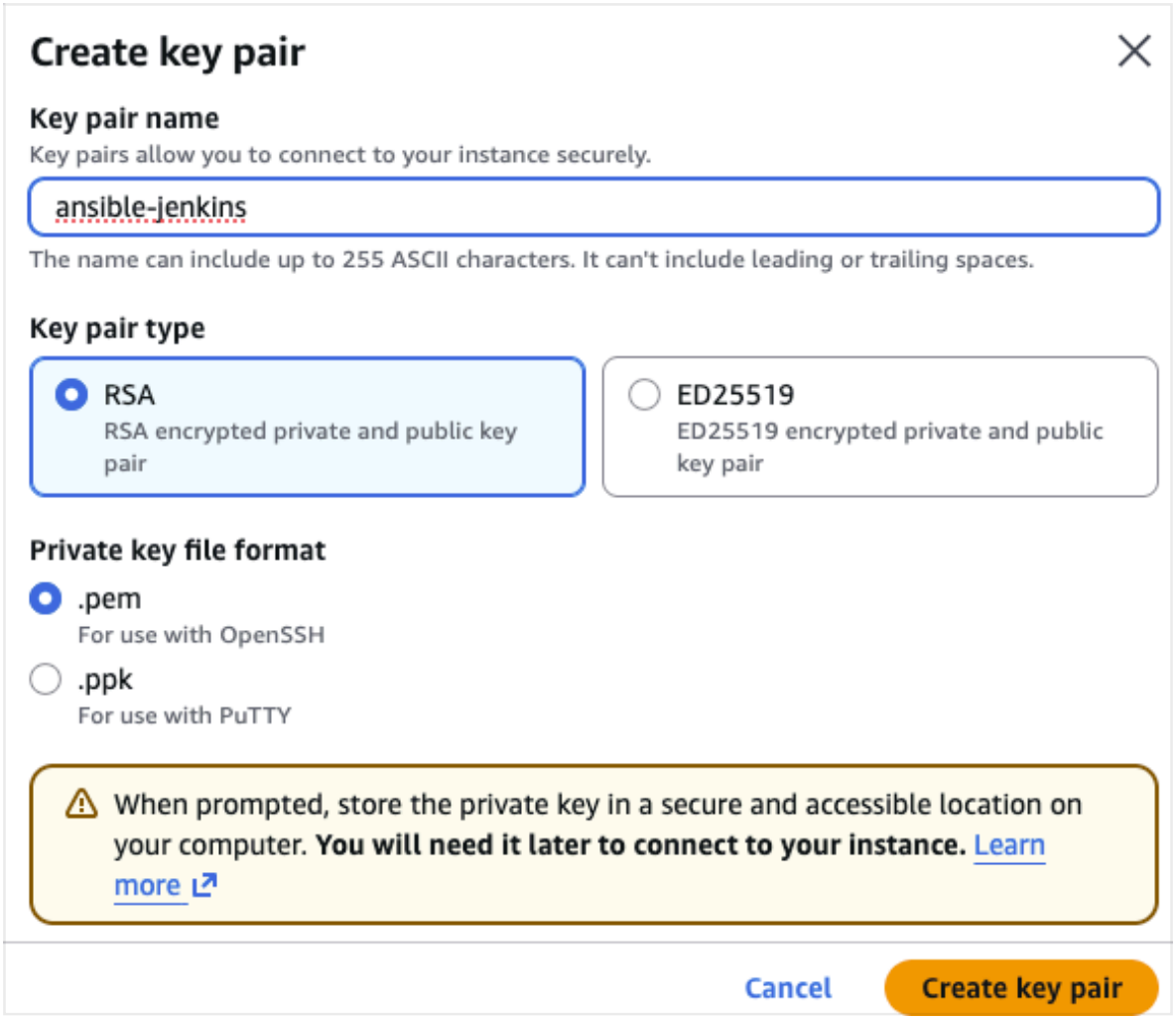
Ansible server is ready ❤️

Create two EC2 Instances to be managed by Ansible

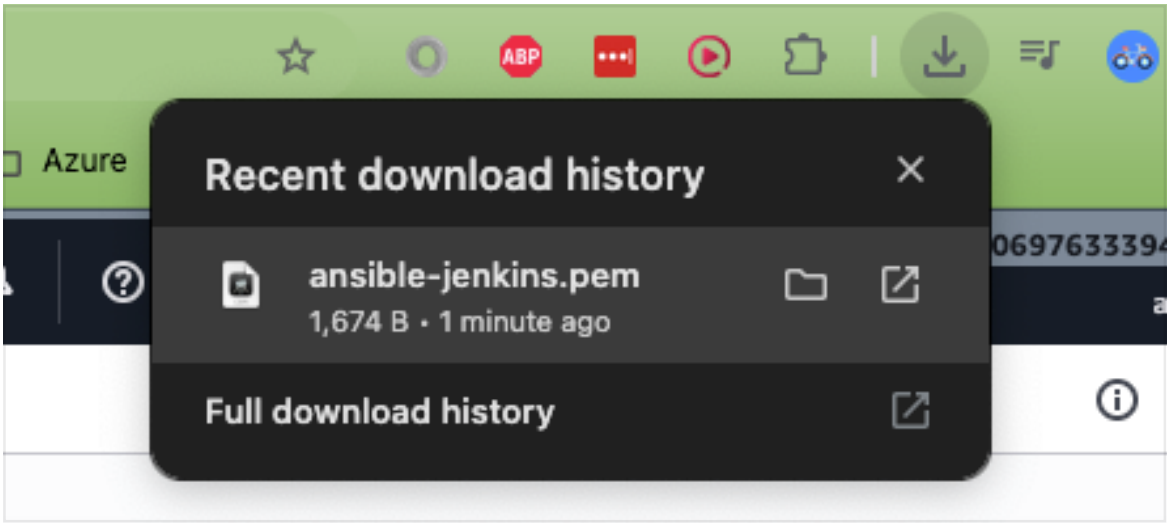
We will launch the EC2 instances manually from the AWS console.



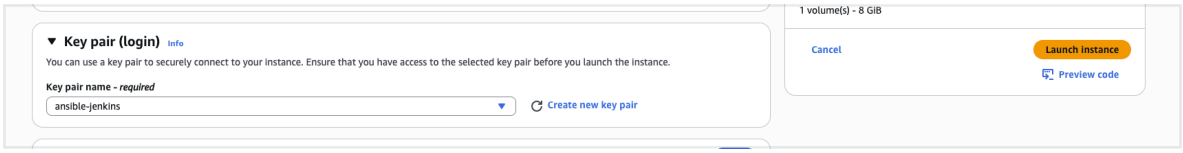
And we will create a new key pair, remember that ansible connects to the instances using the ssh key

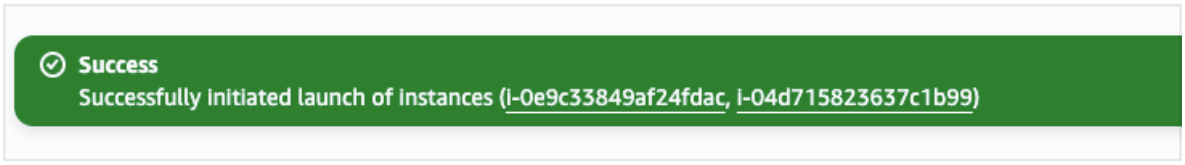


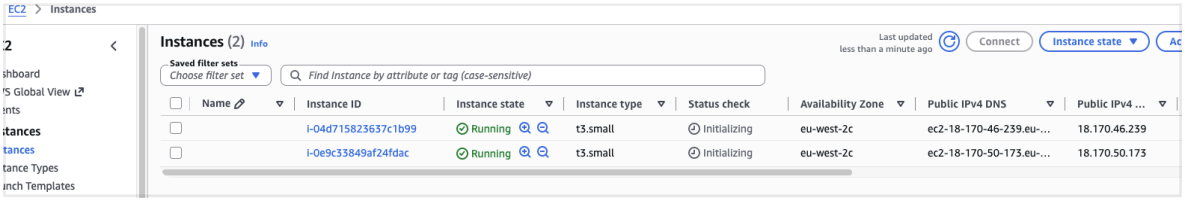
And we download the .pem file



And we launch the 2 instances







So we have two EC2 instances and the private key to connect to those two instances.

This is all we need to configure for the EC2 instances, the rest of the configuration will be done by Ansible.